

Section 2: Control Path Operation

EEP1 Jump instructions

EEP1 machine code	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	1	1	0	0	JMPOPC				Imms8 (JOFFSET)							
JMPOPC(3:1)	NZCV condition				Condition on data for jump with JMPOPC(0)=0				JMPOPC(0) = 0				JMPOPC(0) = 1 ³			
0	1				always				JMP (Always jump)				NOP (Never jump)			
1	Z				result = 0				JEQ (Equal)				JNE (Not Equal)			
2	C				$C=1 \equiv u(Ra) \geq u(Op)^1$				JCS (Carry Set / Unsigned Higher or Equal)				JCC (Carry Clear / Unsigned Lower)			
3	N				$z(result) < 0$				JMI (Minus)				JPL (Plus)			
4	$\bar{N} \oplus V$				$z(Ra) \geq z(Op)^1$				JGE (Signed Greater or Equal)				JLT (Signed less than)			
5	$(\bar{N} \oplus V), Z$				$z(Ra) > z(Op)^1$				JGT (Signed Greater Than)				JLE (Signed Less than or Equal)			
6	C, Z				$u(Ra) > u(Op)^1$				JHI (Unsigned Higher)				JLS (Unsigned Lower or Same)			
7	1				always				JSR ($R7^2 := PC + 1, PC := PC + JOFFSET$)				RET ($PC := R7$)			

¹ JGE, JGT, JHI, JCS jump conditions jump based on the flag values. The table column 3 shows the comparison of Ra and Rb after a previous SUB Ra, Rb instruction which delivers those NZCV values, or the condition on an ALU result which delivers the required NZ values.

² JSR and RET always jump and are used to jump to and return from subroutine see notes. RET does not use Joffset.

³ JMPOPC(0) = 1 inverts all conditions, except for JSR/RET

⁴ Hardware is simpler because JSR/RET write/read R7 since Ra is normally read and written and for these instructions a = JMPOPC(3:1) = 7

Jump Instructions	Assembly	Operation	Examples
All except RET, JSR	JEQ JOFFSET; JEQ SYMBOL	PC := PC + JOFFSET if condition is true	JMP -1 ; JGT LOOP ; JSR SUB1
RET		PC := R7	RET
JSR	JSR JOFFSET	R7 := PC+1; PC := PC + JOFFSET	JSR 10

Figure 2: EEP1 Jump Instructions

- Objectives
- In this section you will:
- examine how the program counter (PC) selects the next instruction,
 - understand how flags are stored and used for conditional jumps,
 - relate control-path signals to the NEXT block behaviour.

Program Counter

Figure 3 shows the schematic of the controlpath block, which contains the EEP1 PC. The output PC.Q is connected to the instruction memory address MEMADDR, and therefore selects which instruction word MEMDATA = INS(15:0) is currently being executed.

The input PC.D must carry the address of the next instruction. This input is driven by NEXT.PCNEXT, meaning that the NEXT block decides whether the control path performs a normal sequential increment or a jump.

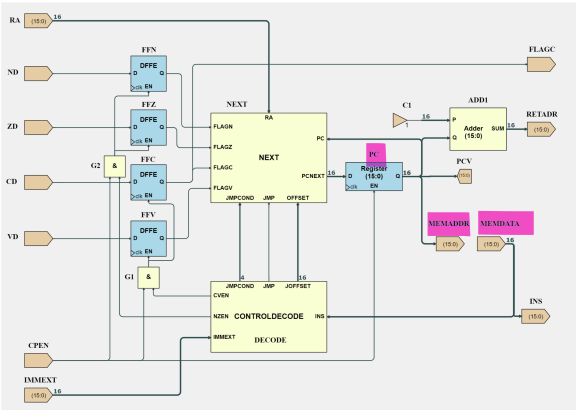
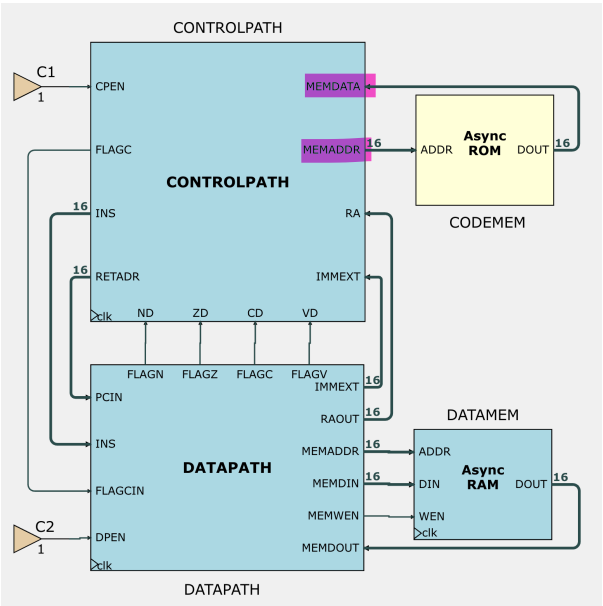


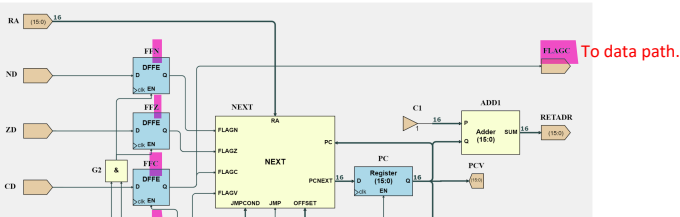
Figure 3: EEP1 Control Path



MEMADDR inputs into ROM and chooses correct operation code for MEMDATA and sends MEMDATA back to Control path.

Flags

The control path also contains four 1-bit flip-flops: FFN, FFZ, FFC, FFV. Their Q outputs form the four EEP1 flags FlagN, FlagZ, FlagC, FlagV, which are inputs to the NEXT block. In addition, FlagC is forwarded to the datapath where it may be used as a carry-in for ALU operations.



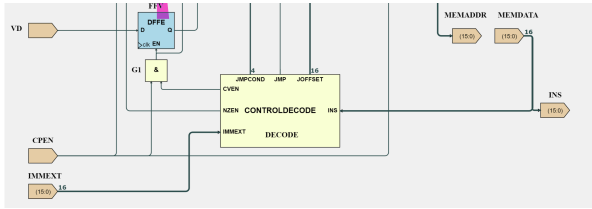


Figure 3: EEP1 Control Path

The block controldecode produces the signals NZEN and CVEN, which determine, for the current instruction, whether (FlagN,FlagZ) or (FlagC,FlagV) should be updated. The ALU instruction specification defines which instructions update which flags (for example, MOV updates only FlagN and FlagZ, whereas ADD updates all four flags).

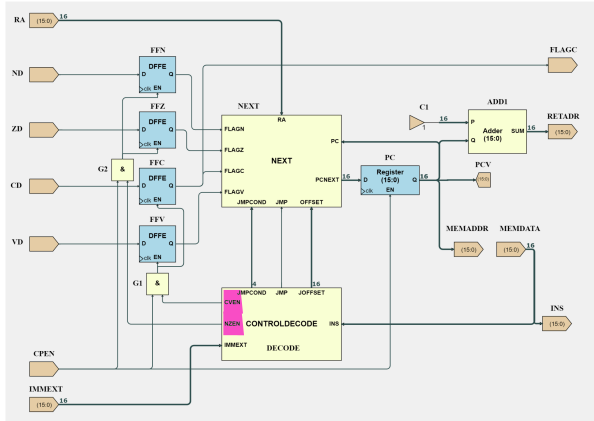


Figure 3: EEP1 Control Path

NZEN & CVEN writes into EN of flags accordingly.

2.1 NEXT block

Figure 4 shows the NEXT sheet. The logic that controls PCNEXT includes multiplexers MUX1 and MUX2, driven by the control signals COND.RET, COND.JUMP, and NEXT.JMP.

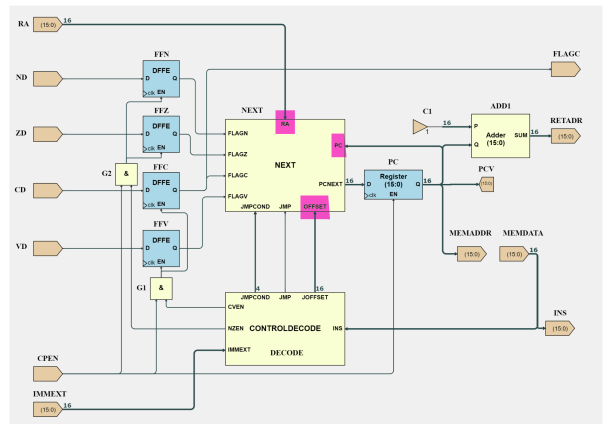
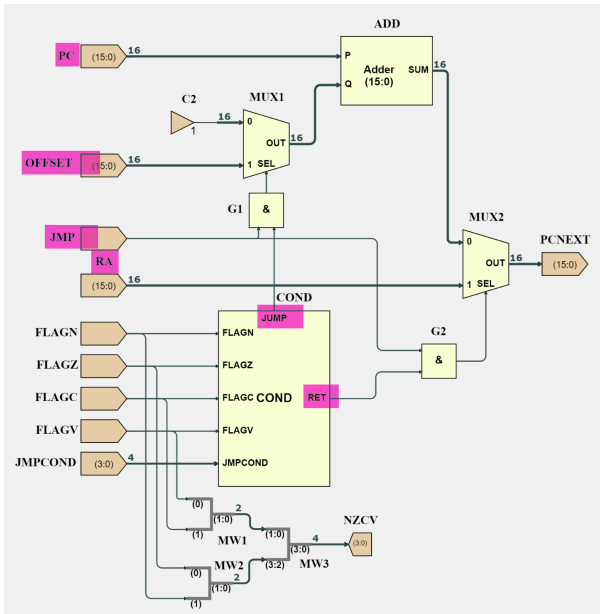


Figure 3: EEP1 Control Path

Draw by hand an algebraic truth table showing how COND.RET, COND.JUMP, and NEXT.JMP, together with the algebraic inputs NEXT.PC, NEXT.OFFSET, and NEXT.RA, determine the value of PCNEXT.

	A	B	C	D	E	F	G	H	I	J
1	NEXT.JMP	COND.JUMP	COND.RET	PCNEXT			NEXT.JMP	COND.JUMP	COND.RET	PCNEXT
2		0	0	0 PC+1				0 X	X	PC+1
3		0	0	1 PC+1				1	0	0 PC+1
4		0	1	0 PC+1				1 X		1 RA
5		0	1	1 PC+1				1	1	0 PC+OFFSET
6		1	0	0 PC+1						
7		1	0	1 RA						
8		1	1	0 PC+OFFSET						
9		1	1	1 RA						



Seen from truth table, we can clearly understand that NEXT.JMP determines whether need a Jump Instruction;
COND.JUMP determines whether our flags satisfy the condition(s);
COND.RET determines whether we return, which is to the address stored in R7.

Task 1. Create an 8-row ISSIE algebraic truth table, selecting components as shown in Figure 5, to examine how the NEXT sheet drives NEXT.PCNEXT. Compare the truth table with your hand-generated version; they should match. You may assume that NEXT.JMP = 1 only for jump instructions. What is the value of PCNEXT when NEXT.JMP = 0? Compare your results with the EEPT1 jump instructions in Figure 2 and explain how the signals COND.JUMP and COND.RET influence the control path behaviour.

G1_IN1	G2_IN1	PC	OFFSET	JMP	RA	PCNEXT
0	0	PC	OFFSET	0	RA	(PC + 1)
0	1	PC	OFFSET	0	RA	(PC + 1)
1	0	PC	OFFSET	0	RA	(PC + 1)
1	1	PC	OFFSET	0	RA	(PC + 1)
0	0	PC	OFFSET	1	RA	(PC + 1)
0	1	PC	OFFSET	1	RA	RA
1	0	PC	OFFSET	1	RA	(OFFSET + PC)
1	1	PC	OFFSET	1	RA	RA

Where we can see that the truth table in ISSIE shows the same result as expected, where
G1_IN1 is COND.JUMP;
G2_IN1 is COND.RET.